Courier Delivery Route and Tracking

K. Moksha Sai 2620030147
T. Varshitha 2620030033



Abstract

A Java system records courier ID, customer name, source, destination, route, and delivery status for a neighborhood pharmacy order.
The workflow moves through Booked, In Transit, Out for Delivery, and Delivered, reducing manual work and making customer lookup clearer.
Future scope: GPS tracking, database connectivity, route optimization, notifications, and a GUI.



Problem Statement

A neighborhood pharmacy may handle many medicine deliveries at once.
Staff must capture customer, source, and destination details, assign a route, update parcel status, and locate a delivery quickly.
Manual records can cause missed updates, delayed deliveries, and slow customer lookup.
The system organizes each delivery record so the courier can move from assignment to confirmation.



CO 1 / Module 1 — Java Foundations

Booking input: Scanner reads each courier ID, customer, and route; variables hold the record, and printf formats one delivery line for the pharmacy order.

Key concepts

- Computational thinking: break booking into steps.
- Algorithm/flowchart: map input, record, and output.
- Java structure: class, main, and statements.
- Primitive types, variables, and constants hold record data.
- Operators and casting process courier details.
- Scanner reads ID and route; printf formats the record.

CO 2 / Module 2 — Selection and Iteration

Route tracking: if/else and switch move a parcel from Booked to In Transit, Out for Delivery, and Delivered; for loops walk route stops, and continue skips completed stops.

Key Java Concepts

- if/else: choose the next action for a parcel marked Booked, In Transit, or Delivered.
- switch with case, default, and break: map each status to one update; default logs unknown statuses.
- while, do-while, and for: for walks route stops; while waits for scans; do-while runs at least once.
- continue: skip a stop already delivered and move to the next pending stop.
- Nested loops: the outer loop visits each route, the inner loop checks each stop's status.

CO 3 / Module 3 — Methods, Arrays, and Search

Dispatch methods: addCourier, updateStatus, and findCourier isolate each courier action; arrays and linear search locate a courier by ID along the route.

Key Concepts

- Modular programming: methods isolate courier actions such as add, update, and find.
- Method definition and invocation: call findCourier(couriers, id) to reuse one lookup routine.
- Parameters and return values: pass the list and ID in, return the matching courier or null.
- Overloading: reuse one name for different parameter lists, such as find by ID or by name.
- Enhanced for loop, recursion basics, and 1D arrays: scan couriers and route stops; linear search, counting, and averages summarize deliveries.

CO 4 / Module 4 — OOP and Exceptions

Courier objects: each parcel is one object with a private ID and a status changed only through update(); exceptions reject invalid status values during tracking.

Key concepts

- Class/object: Courier models one delivery.
- Fields/methods: id, status; update() and toString().
- Constructor sets status; this.id names the field.
- Private fields: getters/setters protect data; static is shared.
- Inheritance and interfaces (Trackable) support reuse.
- Exceptions reject invalid IDs or status values.

CO 5 / Module 5 — Strings, Files, and Records

Delivery records: split() parses pharmacy order fields, Files reads deliveries.txt, and sorting organizes pending deliveries by courier ID.

Key concepts

- Strings are immutable; use equals() for content and == only for object identity.
- split("\\|"), join() and trim() parse delivery-record fields.
- StringBuilder builds report lines efficiently.
- Sorting and searching use Comparable or Comparator on delivery IDs.
- Path and Files with buffered I/O read and write deliveries.txt.
- Malformed records are checked; try-with-resources closes files safely.

CO 6 / Module 6 — Collections and Streams

Live tracking: a HashMap links courier IDs to statuses, and streams filter Out for Delivery parcels into a report for the dispatch team.

Core Concepts

- List preserves delivery order; Set removes duplicate stops; Map links IDs to statuses.
- ArrayList, HashSet, and HashMap implement collections; generics keep types safe.
- Lambdas express rules: Predicate filters, Function transforms, Consumer acts.
- Streams filter, map, sort, and distinguish delivery data, then collect results.
- groupingBy, counting, and joining summarize delivery reports.

Conclusion

The six modules work together as one maintainable courier route and tracking system.
Java foundations capture the pharmacy order; control flow updates status; methods and arrays manage actions; OOP models couriers; strings and files organize records; collections and streams produce live reports.
Future scope: GPS tracking, database connectivity, route optimization, notifications, and a GUI.



Java Program Screenshots

Courier input (Scanner + printf)

```
Scanner sc = new Scanner(System.in);
String courierId = sc.nextLine();
String customer = sc.nextLine();
System.out.printf("%s | %s | %s to %s | %s%n",
 courierId, customer, source, destination, status);
```

Delivery status switch

```
switch (status) {
case "Booked": status = "In Transit"; break;
case "In Transit": status = "Out for Delivery"; break;
case "Out for Delivery": status = "Delivered"; break;
default: System.out.println("Status recorded");
}
```

Courier class

```
public class Courier {
 private final String id;
 private String status;
public Courier(String id) { this.id = id; this.status = "Booked"; }
 public void update(String s) { status = s; }
@Override public String toString() { return id + " / " + status; }
}
```

File parsing and stream report

```
String[] fields = line.split("\\|");
try (var lines = Files.lines(Path.of("deliveries.txt"))) {
lines.map(String::trim).sorted().forEach(System.out::println);
}
live.entrySet().stream()
.filter(e -> e.getValue().equals("Out for Delivery"))
.map(Map.Entry::getKey).sorted().collect(Collectors.toList());
```

Console status C17 | Booked → In Transit → Out for Delivery → Delivered

Geo-Tag Photo and GitHub Link

